

Memória virtual, memória compartilhada e comunicação entre processos

Disciplina: Sistemas Operacionais

Professor: Michael Douglas C Alves

Alunos: Vitor Hugo, Thiago Wyse e Matheus Bertemes

Data: 01/10/2026

1. Objetivo

Implementar dois processos independentes em C++ para Windows: um produtor que calcula a sequência de Collatz e a grava em memória compartilhada, e um consumidor que lê e apresenta os valores. A sincronização deve impedir a leitura de dados incompletos e manter a memória disponível até a confirmação do consumidor.

2. Fundamentação teórica

Um programa é um conjunto de instruções armazenado. Um processo é uma instância em execução, com estado, recursos e espaço de endereçamento. Duas instâncias podem executar o mesmo código mantendo dados privados diferentes.

A memória virtual separa os endereços usados pelo programa da organização física da RAM. Na paginação, o espaço virtual é dividido em páginas e a memória física em molduras, ou frames. A tradução de endereços relaciona páginas virtuais a molduras físicas. Uma página necessária que não esteja residente pode causar uma falta de página, tratada pelo sistema operacional; isso não implica necessariamente acesso a disco.

A comunicação entre processos, ou IPC, permite a troca de informações entre espaços normalmente isolados. A memória compartilhada disponibiliza o mesmo armazenamento em vistas de processos diferentes, preservando a privacidade das demais regiões. As tabelas de páginas permitem que endereços virtuais distintos correspondam ao mesmo armazenamento [1].

A Conjectura de Collatz propõe que, para qualquer inteiro positivo, aplicar $n/2$ quando n é par e $3n+1$ quando n é ímpar termina em 1. O programa implementa essas regras, sem demonstrar a conjectura. O argumento recebido é o valor inicial, não o número de elementos a produzir.

3. Implementação

Código-fonte: <https://github.com/thiagowyseoficial/atividade-so-collatz>. Os arquivos principais são produtor.cpp, consumidor.cpp e compartilhado.h.

3.1. Organização e limites

```
struct DadosCompartilhados {  
    std::uint64_t quantidade;  
    std::uint64_t valores[4096];  
};
```

O tamanho lógico solicitado é $8 + 4096 \times 8 = \mathbf{32.776 \text{ bytes}}$, verificado por `static_assert`. O gerenciamento físico usa páginas; esse valor não corresponde necessariamente à RAM residente exata. A quantidade inclui o valor inicial e o 1 final. O consumidor só aceita quantidades de 1 a 4096 e ignora posições não utilizadas.

O tipo `uint64_t` representa de 0 a 18.446.744.073.709.551.615. A entrada deve conter apenas dígitos e ser positiva. O produtor verifica overflow na conversão e antes de $3n+1$. Sequências acima de 4096 elementos são rejeitadas antes da publicação. Não existem ponteiros no bloco compartilhado, pois endereços de um processo não são diretamente utilizáveis pelo outro.

3.2. Produtor e consumidor

O produtor valida o argumento e calcula a sequência em um vetor privado. Cria dois eventos e um objeto nomeado com `CreateFileMappingW`. O uso de `INVALID_HANDLE_VALUE` faz o objeto ter respaldo no arquivo de paginação do sistema [2]; não há arquivo de dados como canal principal. Depois de `MapViewOfFile`, copia os valores, grava a quantidade e sinaliza o evento de dados prontos.

O consumidor abre o mesmo objeto com `OpenFileMappingW`, mapeia uma vista somente de leitura e abre os eventos com `OpenEventW`. Aguarda a publicação, valida a quantidade, imprime os valores e confirma a leitura. Ele não contém o algoritmo de Collatz. Ambos exibem PID e endereço virtual da vista para observação.

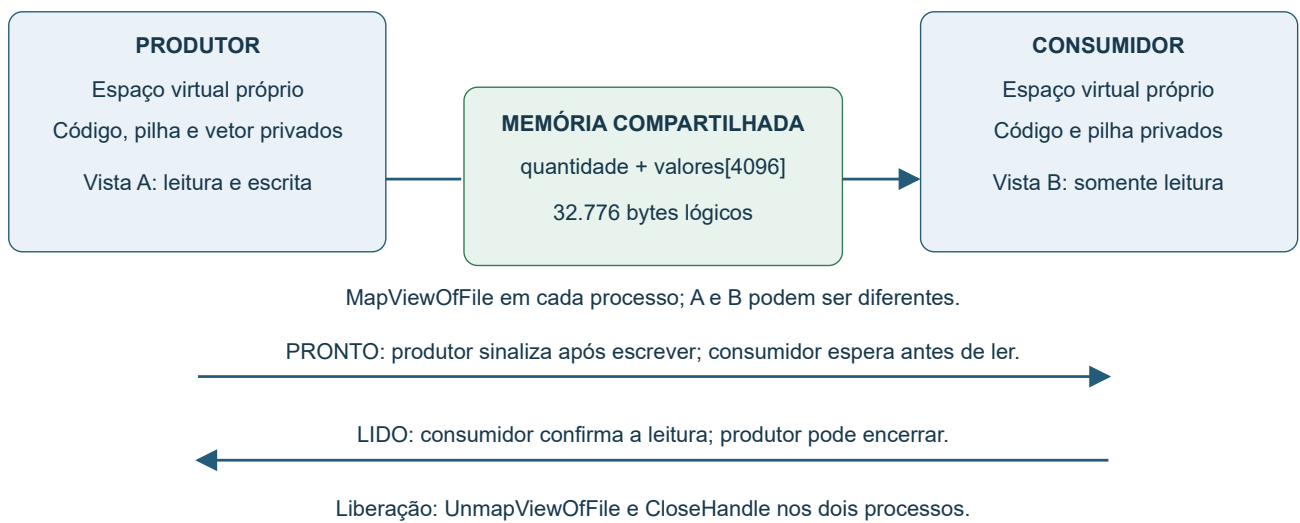
3.3. Sincronização e recursos

Os eventos Pronto e Lido são de reset manual e começam não sinalizados. O produtor sinaliza Pronto somente depois de concluir todas as escritas. O consumidor espera Pronto, lê e sinaliza Lido. O produtor espera Lido antes de liberar seus recursos. O sinal manual permanece disponível se o consumidor iniciar a espera após a publicação [3]. As funções de sinalização e espera do Windows também fornecem a ordenação adequada da memória [4].

O projeto admite um produtor e um consumidor por rodada, na mesma sessão do Windows, com nomes no namespace `Local\`. Eventos são criados antes do mapeamento. Objetos existentes são rejeitados para evitar mistura de rodadas. Como não há escrita após a publicação, não é necessário um mutex para percorrer o vetor nessa comunicação de publicação única.

Cada espera tem limite de 120 segundos. Falhas de API, timeout e entrada inválida retornam código 1. Em sucesso, ambos retornam 0. Objetos RAII chamam `UnmapViewOfFile` e `CloseHandle`, inclusive ao sair por exceção. A confirmação mantém o produtor vivo para que o consumidor consiga abrir o objeto.

4. Arquitetura



O fluxo de dados é produtor → memória compartilhada → consumidor. A e B são endereços virtuais simbólicos. As vistas podem começar em posições diferentes, enquanto o objeto compartilhado e os deslocamentos permanecem comuns.

5. Compilação, testes e resultados

A validação foi executada no GitHub Actions, em **Windows Server 2022 Datacenter**, arquitetura **x64**. Data/hora UTC: 2026-10-01T23:17:06.204Z. O Windows local havia bloqueado os executáveis; os resultados abaixo foram obtidos efetivamente no ambiente Windows remoto.

Execução verificável: <https://github.com/thiagowyseoficial/atividade-so-collatz/actions/runs/36939906782>

Commit testado: 180bce14e27235806f719514f57bb52dac328a0e.

5.1. Compilação

O script `compilar.bat` localiza o Visual Studio, ativa o ambiente x64 e compila com C++17, /W4 /WX /O2 /MT /utf-8. Os dois programas foram gerados com código de saída 0. Avisos são tratados como erros. O log integral está no Apêndice A e em `evidencias/compilacao.txt`.

5.2. Três testes obrigatórios

Para cada entrada, foram iniciados dois executáveis distintos: `produtor.exe` e `consumidor.exe`. A automação aguardou a mensagem de publicação do produtor antes de iniciar o consumidor. Os dois processos encerraram com código 0 e o produtor confirmou a leitura. As sequências completas e quantidades foram comparadas com as referências.

Entrada	Sequência observada	Quantidade	Resultado
5	5 16 8 4 2 1	6	Aprovado
10	10 5 16 8 4 2 1	7	Aprovado
13	13 40 20 10 5 16 8 4 2 1	10	Aprovado

As saídas integrais, incluindo comandos, PIDs, endereços e códigos de saída, estão no Apêndice B e nos arquivos `evidencias/teste_5.txt`, `teste_10.txt` e `teste_13.txt`.

5.3. Casos adicionais e limites da validação

Foram aprovadas **14 verificações**: as três entradas obrigatórias, a entrada 1, a rejeição de um segundo produtor simultâneo, oito casos de entrada inválida e o consumidor sem produtor. As entradas inválidas cobrem argumentos ausentes ou em excesso, zero, negativo, texto, decimal, valor acima de `uint64_t` e overflow de `3n+1`. O resumo estruturado está em `evidencias/resultados.json`.

Os testes confirmam comunicação real entre processos e leitura dos valores publicados. Não constituem prova de ausência de todas as condições de corrida. O consumidor iniciado antes da publicação, o timeout de 120 segundos e o limite de capacidade de 4096 elementos não foram exercitados automaticamente. A ordem de escrita e sinalização foi também revisada no código.

5.4. Reprodução manual

Compile com `compilar.bat`. Em um terminal, execute `.\produtor.exe` 5. Após a mensagem de dados prontos, execute `.\consumidor.exe` em outro terminal, em até 120 segundos. Repita com 10 e 13, aguardando o encerramento dos dois programas entre rodadas. Para repetir as 14 verificações automáticas, execute `node testar.cjs` em Windows com Node.js instalado.

6. Questões conceituais

1. O que é memória virtual e qual problema ela procura solucionar?

É a abstração que separa os endereços usados pelo programa da localização física dos dados. Facilita isolamento, proteção, alocação e compartilhamento controlado, permitindo que nem todo o conteúdo virtual esteja simultaneamente na RAM. Não se resume a usar disco como extensão da memória.

2. O que é um espaço de endereçamento virtual?

É o conjunto de endereços virtuais que um processo pode utilizar. Contém regiões de código, pilha, heap, bibliotecas e mapeamentos, além de regiões ainda não alocadas. Um endereço virtual não é diretamente um endereço físico.

3. Por que dois processos possuem, normalmente, espaços de endereçamento independentes?

Para proteger seus dados e permitir que cada um execute sem modificar acidentalmente a memória do outro. A separação limita as consequências de acessos inválidos e permite ao sistema aplicar permissões.

4. Se o produtor possuir `int valor = 10`, por que o consumidor não pode simplesmente acessar essa variável?

A variável está em uma região privada do produtor. Seu endereço é interpretado no contexto desse processo. O mesmo número de endereço no consumidor pode apontar para outros dados ou para uma região não mapeada. É necessário comunicar o valor por um mecanismo de IPC.

5. Qual é a função da memória compartilhada nesta atividade?

Disponibilizar a quantidade e os valores da sequência ao consumidor. O produtor escreve no bloco e o consumidor lê esse mesmo armazenamento por uma vista em seu próprio espaço virtual.

6. Qual a diferença entre memória privada de um processo e memória compartilhada?

A privada não fica diretamente disponível aos demais processos. A compartilhada é uma região explicitamente acessível a mais de um processo, com permissões determinadas. Havendo escrita, o acesso precisa ser coordenado.

7. Qual é a função de `CreateFileMapping()`?

Criar ou abrir um objeto de mapeamento. O produtor solicita 32.776 bytes e usa `INVALID_HANDLE_VALUE`, fazendo o objeto ter respaldo no arquivo de paginação, sem um arquivo de dados como canal de comunicação. A função retorna um handle, não o ponteiro para os dados.

8. Qual é a função de MapViewOfFile()?

Mapear uma vista do objeto no espaço virtual do processo chamador e retornar o endereço dessa vista. O produtor solicita acesso com escrita e o consumidor solicita somente leitura.

9. Por que o consumidor também precisa mapear a região de memória?

Abrir o objeto fornece um handle, mas não cria um ponteiro utilizável pelo consumidor. É necessário mapear uma vista no espaço virtual dele para acessar os bytes compartilhados.

10. Qual é a relação entre memória virtual e o mapeamento de uma região de memória compartilhada?

O sistema associa regiões virtuais dos processos ao mesmo armazenamento. Assim, eles continuam independentes, mas passam a compartilhar especificamente o bloco mapeado.

11. Por que produtor e consumidor precisam ser sincronizados?

Eles podem executar em velocidades e momentos diferentes. A leitura deve começar somente depois de o produtor concluir e publicar todas as escritas. A existência do mapeamento, sozinha, não indica que a sequência está pronta.

12. O que pode acontecer se o consumidor ler a memória antes de o produtor terminar de gravá-la?

Ele pode observar zeros iniciais, uma quantidade ainda não publicada ou apenas parte da sequência. Um protocolo incorreto pode até causar leitura fora dos limites. Nesta solução, a espera pelo evento e a validação da quantidade impedem a leitura prematura e limitam o acesso ao vetor.

13. Qual mecanismo de sincronização foi utilizado? Explique seu funcionamento.

Dois eventos nomeados de reset manual, inicialmente não sinalizados. O produtor sinaliza Pronto depois de escrever. O consumidor aguarda Pronto, lê e sinaliza Lido. O produtor aguarda Lido antes de encerrar. Cada rodada usa objetos novos; uma segunda sessão simultânea é rejeitada.

14. Qual é a função de WaitForSingleObject() na solução?

Suspender a thread até o evento ser sinalizado ou até expirar o limite de 120 segundos. O consumidor espera os dados; o produtor espera a confirmação. O código diferencia WAIT_OBJECT_0, WAIT_TIMEOUT e WAIT_FAILED, sem laço de espera ocupada.

15. Como produtor e consumidor podem utilizar endereços virtuais diferentes para acessar a mesma região de memória compartilhada?

Cada processo possui sua própria tradução de endereços. A página virtual de um processo e outra página virtual do segundo podem apontar para o mesmo armazenamento compartilhado. Os ponteiros não precisam ter o mesmo valor: o objeto e o deslocamento dos dados é que precisam corresponder.

7. Questão central e conclusão

Dois processos independentes acessam os mesmos dados quando o sistema operacional mapeia o mesmo objeto compartilhado nos respectivos espaços virtuais. A independência das regiões privadas permanece; somente o bloco disponibilizado participa da comunicação. Os eventos organizam a publicação, a leitura e o encerramento.

A implementação reuniu geração de Collatz, memória compartilhada nomeada, sincronização e liberação de recursos. A compilação e as 14 verificações foram concluídas no Windows do GitHub Actions. Os três exemplos obrigatórios produziram as sequências esperadas e terminaram normalmente nos dois processos. A demonstração em sala ainda deve ser ensaiada no computador que será utilizado.

8. Referências

- [1] Microsoft Learn. [Virtual Address Space](#).
- [2] Microsoft Learn. [CreateFileMappingW](#).
- [3] Microsoft Learn. [Event Objects](#).
- [4] Microsoft Learn. [Synchronization and Multiprocessor Issues](#).
- [5] Microsoft Learn. [WaitForSingleObject](#).

Enunciado da atividade complementar fornecido na disciplina. Documentação consultada em 01/10/2026. A literatura de Silberschatz indicada no enunciado é leitura complementar recomendada; não foi consultada diretamente nesta resolução.

Apêndice A — Evidência de compilação

Compilacao C++17 / Windows x64

COMANDO: cl /std:c++17 /EHsc /W4 /WX /O2 /MT /utf-8 produtor.cpp /Fe:produtor.exe

Microsoft (R) C/C++ Optimizing Compiler Version 19.44.35229 for x64

Copyright (C) Microsoft Corporation. All rights reserved.

produtor.cpp

Microsoft (R) Incremental Linker Version 14.44.35229.0

Copyright (C) Microsoft Corporation. All rights reserved.

/out:produtor.exe

produtor.obj

COMANDO: cl /std:c++17 /EHsc /W4 /WX /O2 /MT /utf-8 consumidor.cpp /Fe:consumidor.exe

Microsoft (R) C/C++ Optimizing Compiler Version 19.44.35229 for x64

Copyright (C) Microsoft Corporation. All rights reserved.

consumidor.cpp

Microsoft (R) Incremental Linker Version 14.44.35229.0

Copyright (C) Microsoft Corporation. All rights reserved.

/out:consumidor.exe

consumidor.obj

RESULTADO: ambos os programas compilados; codigo 0.

Apêndice B — Evidências de execução

Entrada 5

```
COMANDO: produtor.exe 5
PRODUTOR PID=5080 endereco virtual=0000018474310000
Entrada: 5
Elementos gravados: 6
Memoria reservada: 32776 bytes
Dados prontos. Aguardando consumidor (120 segundos)...
Leitura confirmada. Recursos serao liberados.
Codigo produtor: 0

COMANDO: consumidor.exe
CONSUMIDOR PID=5160 endereco virtual=000001E19B190000
Aguardando sinal de dados prontos...
Quantidade: 6
Sequencia: 5 16 8 4 2 1
Leitura concluida. Recursos serao liberados.
Codigo consumidor: 0
```

Entrada 10

```
COMANDO: produtor.exe 10
PRODUTOR PID=7312 endereco virtual=0000024F7F160000
Entrada: 10
Elementos gravados: 7
Memoria reservada: 32776 bytes
Dados prontos. Aguardando consumidor (120 segundos)...
Leitura confirmada. Recursos serao liberados.
Codigo produtor: 0

COMANDO: consumidor.exe
CONSUMIDOR PID=7412 endereco virtual=0000025372070000
Aguardando sinal de dados prontos...
Quantidade: 7
Sequencia: 10 5 16 8 4 2 1
Leitura concluida. Recursos serao liberados.
Codigo consumidor: 0
```

Entrada 13

```
COMANDO: produtor.exe 13
PRODUTOR PID=3296 endereco virtual=0000028048B60000
Entrada: 13
Elementos gravados: 10
Memoria reservada: 32776 bytes
Dados prontos. Aguardando consumidor (120 segundos)...
Leitura confirmada. Recursos serao liberados.
Codigo produtor: 0
```

```
COMANDO: consumidor.exe
CONSUMIDOR PID=1548 endereco virtual=0000019179360000
Aguardando sinal de dados prontos...
Quantidade: 10
Sequencia: 13 40 20 10 5 16 8 4 2 1
Leitura concluida. Recursos serao liberados.
Codigo consumidor: 0
```